

Python DS&A Team Project

턴 기반 TUI 로그라이크 — Dungeon Crawler

Team 25 · Jinwoo Park

AI2000 · Data Structures and Algorithms

Contents

1. 프로젝트 개요
2. Dungeon Map
3. Undo System
4. Turn Management
5. Item Inventory
6. Enemy AI
7. Leaderboard
8. 정리

프로젝트 개요

개요

장르: 턴 기반 TUI 로그라이크 (룸 템플릿 던전 크롤러)

기술 스택

- Python 3.9+
- rich (TUI)
- stdlib + rich only

6 Core Features



Dungeon Map

2D · Graph · A*



Undo System

Stack · Command Pattern



Turn Management

Batch resolve · Turn counter



Item Inventory

List (10 slots)



Enemy AI

A* · min-heap



Leaderboard

Array · Insertion Sort

게임플레이

이동·전투

- w/a/s/d — 한 칸 이동 (적 칸 → 근접 공격)
- c + 방향 — 근접 공격 모드
- v + 방향 — 원거리 발사 (화살 3개 시작)
- 1 0 — 인벤토리 슬롯 아이템 사용

기타

- i — 인벤토리 표시 토글
- u / r — Undo / Redo (턴 단위)
- q / esc — 메인 메뉴 포기

성장·승패

- EXP → 레벨업 (HP/ATK/DEF 증가, 풀회복)
- 이동 시 바닥 아이템 자동 루팅
- 계단 타일 위 → 자동 하강
- 3층 계단 = 승리

Dungeon Map

이동 및 충돌 검사

Core Feature · Dungeon Map

set

장점 사용하는 타일만 기억 → 메모리 절약, Python hash table 기반 빠른 조회

단점 맵 크기(53×30) 표현 불가 → 범위 검사 별도 필요

특징 floor_tiles / _blocked set으로 53×30 월드 상태 관리

2D array

장점 인덱스 접근 $O(1)$, 구현 직관적

단점 전체 1,590칸 항상 할당

특징 바닥이 없는 빈 공간까지 메모리 점유

선택 이유

실제 바닥 타일만 저장하는 sparse 구조가 메모리·로직 모두 효율적

코드 위치 models/dungeon.py – Dungeon.is_walkable()

방 연결 구조

Core Feature · Dungeon Map

Graph (간선 set)

장점 방 $R \leq 12$ 에 경량, 연결 관계 명시적

단점 노드 수 증가 시 edge 관리 필요

특징 room_nodes + edges로 복도 캐빙 대상 결정

인접 행렬

장점 두 방 연결 $O(1)$ 조회

단점 12×12 도 대부분 0, 메모리 낭비

특징 희소 그래프에 과한 표현

선택 이유

층당 12방 규모에서 간선 set이 단순하고 충분

코드 위치 generation/room_graph.py – build_room_graph()

맵 생성 — 방 연결

Core Feature · Dungeon Map

Greedy Spanning Tree

장점 모든 방 최소 연결 보장, $R \leq 12$ 에 단순

단점 최적 트리만으로는 루프 없음 → extra edge 보완

특징 맨해튼 최소 쌍 연결 + extra_cycles=2 순환 간선

BFS / DFS 연결

장점 연결성 검사·탐색에 적합

단점 생성 시 최소 연결 보장 로직이 분리됨

특징 연결 확인용으로는 OK, 생성 파이프라인엔 ST가 직접적

선택 이유

생성 단계에서 한 번에 전 방 연결 + 루프 다양화

코드 위치 generation/room_graph.py – build_spanning_tree()

맵 생성 — 통로 생성

Core Feature · Dungeon Map

A* (맨해튼)

장점 protected로 타 방 관통 방지, 최단 경로

단점 $WH \log WH$ — 맵 크기에 비례

특징 방 외곽 타일 연결 → A* → floor_tiles에 복도 추가

L자 직선 복도

장점 구현 매우 단순

단점 중간에 다른 방 벽·내부 관통

특징 앵커만 있는 직선/꺾은선은 충돌 빈번

선택 이유

L자 직선 복도는 다른 방을 막힌 벽으로 보고 우회하지 않아, 방을 뚫고 통로가 생성되는 문제가 발생하여, 대안으로 A*를 선택

코드 위치 generation/corridor_carver.py — find_corridor_path()

Undo System

행동 되돌리기 (Undo)

Core Feature · Undo System

Two-Stack (LIFO)

장점 push/pop $O(1)$, Undo 순서 자연스러움

단점 히스토리 길이에 비례 메모리

특징 undo_stack / redo_stack, EndTurnAction으로 턴 경계

Queue (FIFO)

장점 순서 보존, 스트림 처리에 적합

단점 Undo는 LIFO인데 FIFO는 순서 역전

특징 되돌리기 UX와 접근 패턴 불일치

선택 이유

마지막 행동부터 되돌리는 게임 규칙과 Stack이 정확히 맞음

코드 위치 engine/undo_manager.py – UndoManager.undo_turn()

Action 기록

Core Feature · Undo System

Command Pattern (`Action` ABC)

장점 추상 클래스로 execute/undo 계약 통일, 원자 단위 delta만 저장

단점 Action 서브클래스·역연산 필드 증가

특징 Action(ABC) — MoveAction · CombatAction · LootAction · ConsumeAction · EndTurnAction

각 execute()가 변경분만 기록, undo()가 역연산

전체 스냅샷 (Memento)

장점 구현 단순, 복원 한 번에

단점 $O(\text{맵} \times \text{턴})$ 메모리

특징 맵·엔티티 전체 복사 부담

선택 이유

행동마다 원자적 Action 객체로 캡슐화하고, ABC가 execute/undo 인터페이스를 강제해

UndoManager는 타입 분기 없이 action.undo()만 호출 — undo 흐름이 통일됨

코드 위치 actions/base_action.py · move/combat/loot/consume_action.py

Turn Management

적 턴 일괄 처리

Core Feature · Turn Management

Batch Action List

장점 플레이어→적 단계 명확, 계획·실행 분리로 로그·Undo 경계 깔끔

단점 공정 순환(라운드 로빈) 미지원

특징 1단계: 적마다 propose → ActionValidator → fallback(≤ 8)
→ list[Action] 수집

2단계: 확정 리스트를 순차 execute + undo_manager.record

FIFO per-entity Queue

장점 다수 액터 공정 순환

단점 플레이어 항상 선행 UX와 충돌

특징 per-entity FIFO 순환 큐는 TUI UX(플레이어 선행)와 맞지 않음

선택 이유

플레이어 1액션 성공 후 run_enemy_turns() — 현재 총 적 전원 행동을

먼저 모두 확정된 뒤 실행해, 턴 단위 Undo·로그가 한 경계로 묶임.

구현의 간단함을 위해 플레이어가 항상 선행하는 방식으로 구현

코드 위치 engine/turn_resolver.py · game_manager.py — run_enemy_turns()

턴 경계 · RNG 동기화

Core Feature · Turn Management

Turn Counter

장점 turn_id O(1), Undo 마커·RNG 시드 통합

단점 속도(AGI) 차등 미표현

특징 `random.seed(base_seed + turn_id)` per turn

Priority Queue

장점 속도 빠른 액터 우선 행동

단점 현재 스탯 모델에 속도 없음, 과잉

특징 턴 순서가 고정된 TUI에 불필요한 복잡도

선택 이유

턴 ID 하나로 RNG 재현 + EndTurnAction 경계를 동시에 해결

코드 위치 `engine/turn_manager.py` – `TurnManager.prepare_turn_rng()`

Item Inventory

슬롯 UI 매핑

Core Feature · Item Inventory

List (10 slots)

장점 키 1~0 ↔ index 1:1, 직관적

단점 빈 슬롯 탐색 $O(10)$

특징 `list[Item | None]` 고정 10칸

Dict only

장점 이름 → 아이템 $O(1)$

단점 슬롯 번호·순서 UI 매핑 어려움

특징 Status 1:♥ 2:- 표시가 불편

선택 이유

인벤토리 UI가 슬롯 번호 기반이라 List가 직접 대응

코드 위치 `models/inventory.py` – `Inventory.slots`

Enemy AI

적 추적 경로

Core Feature · Enemy AI

A* (맨해튼)

장점 최단 경로 보장, 휴리스틱으로 탐색 축소

단점 blocker-fallback 처리 필요

특징 시야 6 이내 플레이어 추적, 4방향 2D

BFS

장점 비가중 2D에서 동일 최단 경로

단점 목표 방향 무시, 탐색 노드 더 많을 수 있음

특징 휴리스틱 없이 전방위 확장

선택 이유

가중치 없는 맵에서 A*+맨해튼은 BFS와 동급이면서 탐색 범위 축소

코드 위치 `utils/pathfinding.py - find_path()`

경로 탐색 우선순위

Core Feature · Enemy AI

min-heap (`heapq`)

장점 open set push/pop $O(\log n)$

단점 heap 자료구조 이해 필요

특징 $A^* f=g+h$ 최소 노드부터 확장

정렬 리스트

장점 구현 단순

단점 삽입마다 $O(n)$ 정렬/삽입

특징 맵이 클수록 open set 비용 증가

선택 이유

우선순위 큐가 A^* 의 핵심 — stdlib heapq로 충분

코드 위치 `utils/pathfinding.py` — `_astar()`

Leaderboard

기록 저장

Core Feature · Leaderboard

Array (list)

장점 인덱스 $O(1)$, capacity 10 고정

단점 삽입 시 정렬 별도 필요

특징 ScoreEntry list $\rightarrow$ leaderboard.json

Linked List

장점 삽입 $O(1)$ (위치 알 때)

단점 i 번째 접근 $O(n)$, Top-10에 이점 없음

특징 10개 고정 순위표에 과한 구조

선택 이유

소규모 고정 배열이 저장·표시 모두 단순

코드 위치 `utils/leaderboard.py` – Leaderboard

점수 순위 정렬

Core Feature · Leaderboard

Insertion Sort

장점 $n \leq 10$ 에서 충분·단순, Top-K 자연스러움

단점 일반적으로 $O(n^2)$

특징 삽입 시 score 내림차순, 초과분 truncate

Min-Heap top-K

장점 대규모 n 에서 $O(n \log k)$

단점 $n=10$ 이면 오버엔지니어링

특징 구현·테스트 복잡도만 증가

선택 이유

기록 10개 제한 — Insertion Sort로 코드·성능 모두 충분

코드 위치 `utils/leaderboard.py` - `_insertion_sort_desc()`

정리

DS/알고리즘 한눈에 보기

Core Feature	DS&A (각 1슬라이드)
Dungeon Map	2D Map · Graph · Spanning Tree · A*
Undo System	Two-Stack · Command Pattern
Turn Management	Batch List · Turn Counter
Item Inventory	List (10 slots)
Enemy AI	A* · min-heap
Leaderboard	Array · Insertion Sort

실행 방법

```
docker build -f dungeon_crawler/Dockerfile -t dungeon-crawler .  
docker run -it --rm dungeon-crawler
```

Q&A

감사합니다